Analysis and Design of Algorithms (ADA)
Lab Assignment – 4

Submitted by : Md. Anas khan

Submitted to : Dr. Aarti

Course : B.Tech – CSE (Data Science)

Roll no. : 2401420035

https://github.com/KLAWS5/graph-algo-mini-project--Md.-Anas-khan-

Capstone Assignment: Delivery Route Optimization

INDEX:

- recurrence-based route cost
- greedy parcel selection
- dynamic programming for delivery planning
- Dijkstra and MST for graph-based route building
- TSP using Held-Karp DP
- profiling and matplotlib visualizations

1. Imports and Input Modeling

We define a small logistics network with a warehouse, customer locations, parcel data, and vehicle capacity.

In [5]:

```
import math,time,random
import matplotlib.pyplot as plt
#import networkx as nx

random.seed(7)

coords={'W':(0,0),'A':(2,3),'B':(5,1),'C':(6,4),'D':(8,0),'E':(9,3)}
def dist(u,v):
    x1,y1=coords[u]
    x2,y2=coords[v]
    return round(math.hypot(x1-x2,y1-y2),2)

nodes=list(coords.keys())
dist_matrix={u:{v:dist(u,v) for v in nodes} for u in nodes}

parcels=[
    {'id':'P1','loc':'A','weight':4,'value':40,'ready':8,'due':12,'service':0.3},
    {'id':'P2','loc':'B','weight':3,'value':35,'ready':8,'due':13,'service':0.3},
    {'id':'P3','loc':'C','weight':5,'value':50,'ready':9,'due':14,'service':0.3},
    {'id':'P4','loc':'D','weight':2,'value':24,'ready':10,'due':16,'service':0.3},
    {'id':'P5','loc':'E','weight':6,'value':54,'ready':11,'due':18,'service':0.3}
]

capacity=10
start_time=8.0

print('Locations:',nodes)
print('Vehicle capacity:',capacity)
```

Out [5]:

```
Locations: ['W', 'A', 'B', 'C', 'D', 'E']
Vehicle capacity: 10
```

2. Recurrence-Based Route Cost

We compute the cost of a fixed route recursively.

In [6]:

```
calls=0
def route_cost(route):
    global calls
    calls+=1
    if len(route)<2:
        return 0
    return dist_matrix[route[0]][route[1]]+route_cost(route[1:])

route=['W','A','C','E','W']
calls=0
cost=route_cost(route)
print('Route:',route)
print('Recursive cost:',cost)
print('Recursive calls:',calls)
```

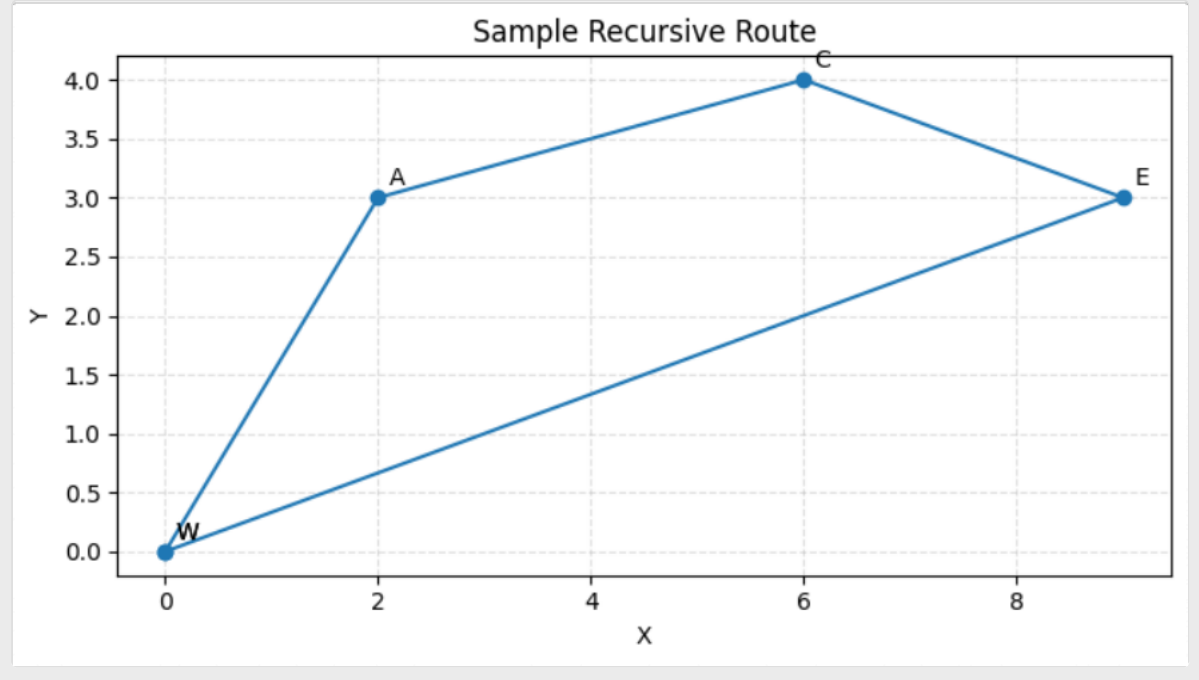
Out [6]:

```
Route: ['W', 'A', 'C', 'E', 'W']
Recursive cost: 20.38
Recursive calls: 5
```

In [7]:

```
fig,ax=plt.subplots(figsize=(7,4))
xs=[coords[n][0] for n in route]
ys=[coords[n][1] for n in route]
ax.plot(xs,ys,marker='o')
for n in route:
    ax.text(coords[n][0]+0.1,coords[n][1]+0.1,n)
ax.set_title('Sample Recursive Route')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.grid(True,linestyle='--',alpha=0.4)
plt.tight_layout()
plt.show()
```

Out [7]:



3. Greedy Parcel Selection

We choose parcels by value-to-weight ratio until capacity is reached.

In [8]:

```
for p in parcels:
    p['ratio']=round(p['value']/p['weight'],2)

greedy=sorted(parcels,key=lambda x:x['ratio'],reverse=True)
picked=[]
w=v=0
for p in greedy:
    if w+p['weight']<=capacity:
        picked.append(p)
        w+=p['weight']
        v+=p['value']

print('Greedy selected parcels:',[p['id'] for p in picked])
print('Total weight:',w)
print('Total value:',v)
```

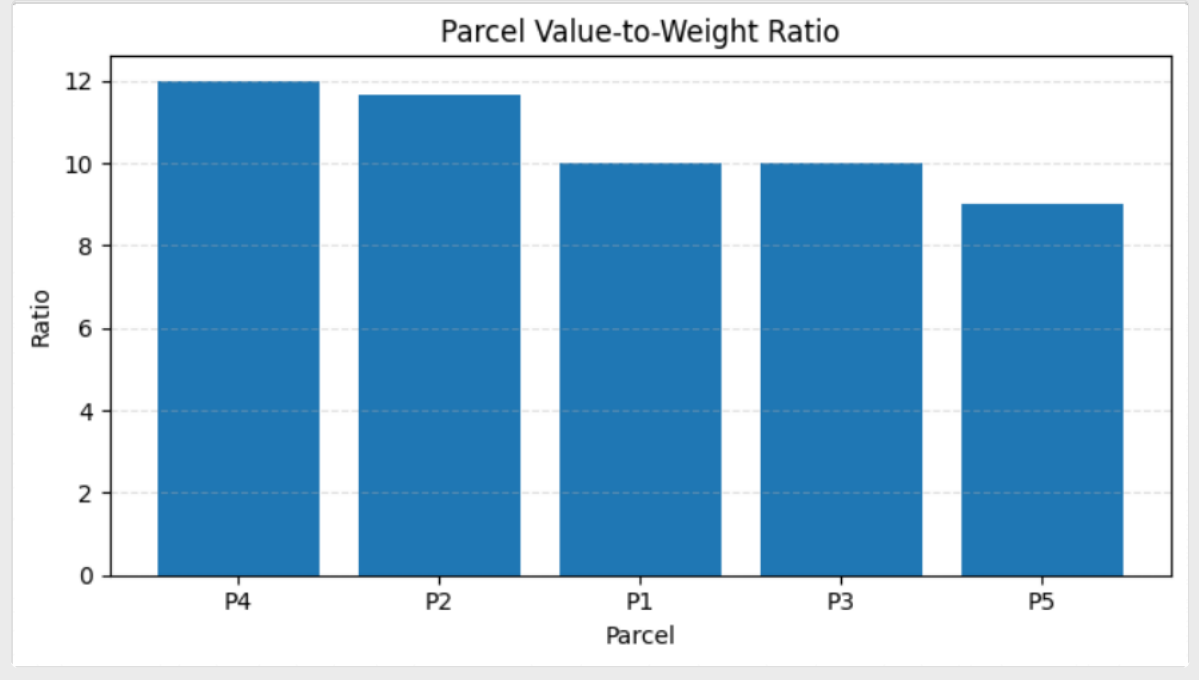
Out [8]:

```
Greedy selected parcels: ['P4', 'P2', 'P1']
Total weight: 9
Total value: 99
```

In [10]:

```
fig,ax=plt.subplots(figsize=(7,4))
labels=[p['id'] for p in greedy]
ratios=[p['ratio'] for p in greedy]
ax.bar(labels,ratios)
ax.set_title('Parcel Value-to-Weight Ratio')
ax.set_xlabel('Parcel')
ax.set_ylabel('Ratio')
ax.grid(True,axis='y',linestyle='--',alpha=0.4)
plt.tight_layout()
plt.show()
```

Out [10]:



4. Dynamic Programming for Delivery Planning

We use 0/1 knapsack DP to maximize parcel value under the vehicle capacity. After choosing the parcels, we validate delivery windows by simulating the route.

In [9]:

```
def knapsack_dp(items,cap):
    n=len(items)
    dp=[[0]*(cap+1) for _ in range(n+1)]
    keep=[[0]*(cap+1) for _ in range(n+1)]
    for i in range(1,n+1):
        wt=items[i-1]['weight']
        val=items[i-1]['value']
        for c in range(cap+1):
            dp[i][c]=dp[i-1][c]
            if wt<=c and dp[i-1][c-wt]+val>dp[i][c]:
                dp[i][c]=dp[i-1][c-wt]+val
                keep[i][c]=1
    chosen=[]
    c=cap
    for i in range(n,0,-1):
        if keep[i][c]:
            chosen.append(items[i-1])
            c-=items[i-1]['weight']
    return list(reversed(chosen)),dp

dp_picked,dp_table=knapsack_dp(parcels,capacity)
dp_w=sum(p['weight'] for p in dp_picked)
dp_v=sum(p['value'] for p in dp_picked)
print('DP selected parcels:',[p['id'] for p in dp_picked])
print('Total weight:',dp_w)
print('Total value:',dp_v)
```

Out [9]:

```
DP selected parcels: ['P2', 'P3', 'P4']
Total weight: 10
Total value: 109
```

In [11]:

```
def simulate_delivery(selected,order=None,start=start_time):
    if order is None:
        order=sorted(selected,key=lambda x:x['due'])
    t=start
    loc='W'
```

```
    rows=[]
    feasible=True
    for p in order:
        travel=dist_matrix[loc][p['loc']]
        t+=travel
        arrive=t
        wait=max(0,p['ready']-t)
        if wait>0:
            t+=wait
        depart=t+p['service']
        ok=(arrive<=p['due']) and (depart<=p['due']+0.5)
        feasible=feasible and ok
        rows.append({'parcel':p['id'],'loc':p['loc'],'arrive':round(arrive,2),'wait':round(wait,2),'depart':round(depart,2),'due':
        t=depart
        loc=p['loc']
    t+=dist_matrix[loc]['W']
    return rows,feasible,round(t,2)

timeline,feasible,end_time=simulate_delivery(dp_picked)
print('Feasible schedule:',feasible)
print('Return-to-warehouse time:',end_time)
for r in timeline:
    print(r)
```

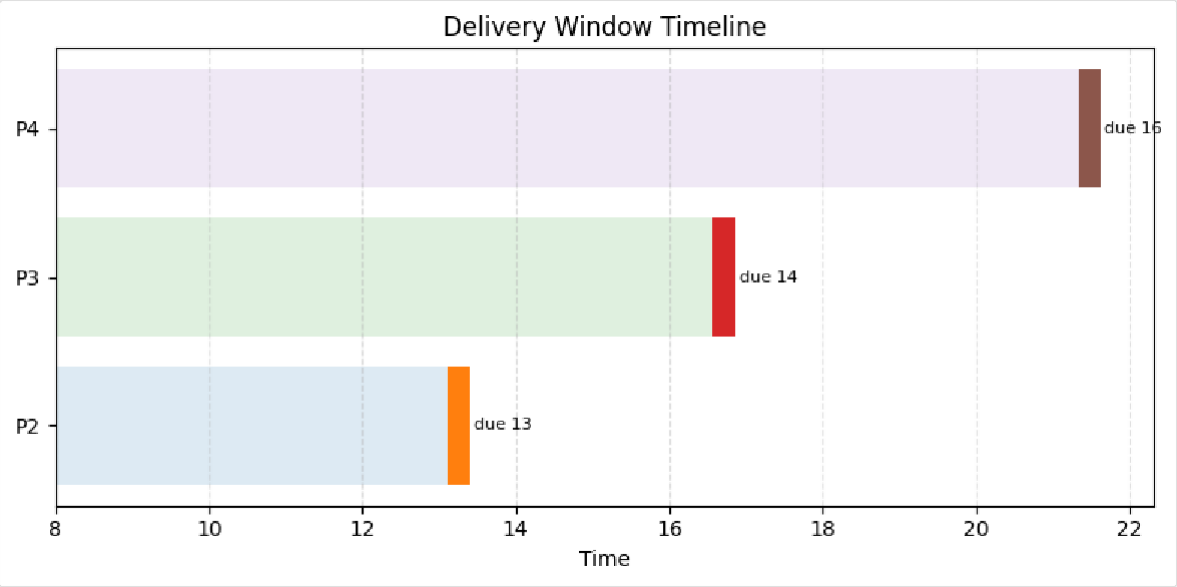
Out [11]:

```
Feasible schedule: False
Return-to-warehouse time: 29.63
{'parcel': 'P2', 'loc': 'B', 'arrive': 13.1, 'wait': 0, 'depart': 13.4, 'due': 13, 'ok': False}
{'parcel': 'P3', 'loc': 'C', 'arrive': 16.56, 'wait': 0, 'depart': 16.86, 'due': 14, 'ok': False}
{'parcel': 'P4', 'loc': 'D', 'arrive': 21.33, 'wait': 0, 'depart': 21.63, 'due': 16, 'ok': False}
```

In [12]:

```
fig,ax=plt.subplots(figsize=(8,4))
for i,r in enumerate(timeline):
    ax.barh(i,r['depart']-start_time,left=start_time,alpha=0.15)
    ax.barh(i,r['depart']-r['arrive'],left=r['arrive'])
    ax.text(r['depart']+0.05,i,f"due {r['due']}",va='center',fontsize=8)
ax.set_yticks(range(len(timeline)))
ax.set_yticklabels([r['parcel'] for r in timeline])
ax.set_xlabel('Time')
ax.set_title('Delivery Window Timeline')
ax.grid(True,axis='x',linestyle='--',alpha=0.4)
plt.tight_layout()
plt.show()
```

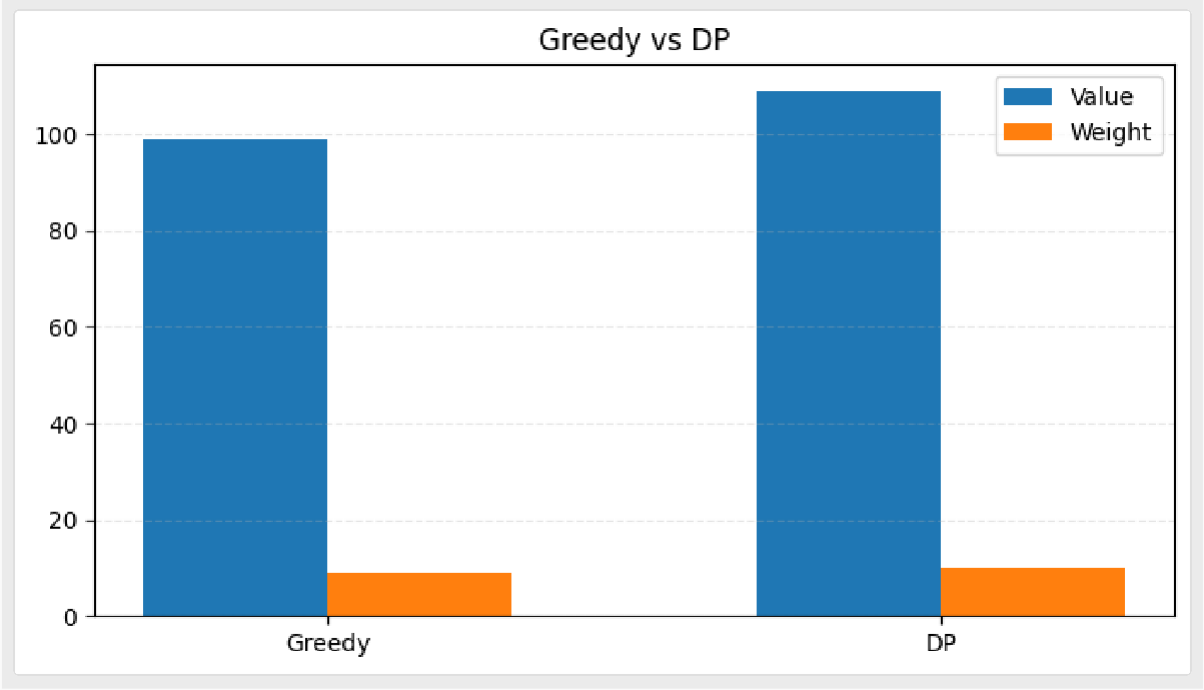
Out [12]:



In [13]:

```
fig,ax=plt.subplots(figsize=(7,4))
methods=['Greedy','DP']
values=[v,dp_v]
weights=[w,dp_w]
x=range(len(methods))
ax.bar([i-0.15 for i in x],values,width=0.3,label='Value')
ax.bar([i+0.15 for i in x],weights,width=0.3,label='Weight')
ax.set_xticks(list(x))
ax.set_xticklabels(methods)
ax.set_title('Greedy vs DP')
ax.legend()
ax.grid(True,axis='y',linestyle='--',alpha=0.4)
plt.tight_layout()
plt.show()
```

Out [13]:



5. Graph Algorithms: Dijkstra and MST

We use the distance graph to compute shortest paths from the warehouse and the minimum spanning tree.

In [14]:

```
graph={u:{v:dist_matrix[u][v] for v in nodes if v!=u} for u in nodes}

def dijkstra(g,start):
    unvisited=set(g)
    d={n:math.inf for n in g}
    prev={n:None for n in g}
    d[start]=0
    while unvisited:
        u=min(unvisited,key=lambda x:d[x])
        unvisited.remove(u)
        for v,w in g[u].items():
            if v in unvisited and d[u]+w<d[v]:
                d[v]=d[u]+w
                prev[v]=u
    return d,prev

def path_to(prev,target):
    path=[]
    while target is not None:
        path.append(target)
        target=prev[target]
    return list(reversed(path))
```

```
shortest,prev=dijkstra(graph,'W')
for n in nodes:
    print(n,round(shortest[n],2),path_to(prev,n))
```

Out [14]:

```
W 0 ['W']
A 3.61 ['W', 'A']
B 5.1 ['W', 'B']
C 7.21 ['W', 'C']
D 8.0 ['W', 'D']
E 9.49 ['W', 'E']
```

In [15]:

```
def prim_mst(g,start):
    visited={start}
    edges=[]
    total=0
    while len(visited)<len(g):
        best=None
        for u in visited:
            for v,w in g[u].items():
                if v not in visited and (best is None or w<best[2]):
                    best=(u,v,w)
        u,v,w=best
        visited.add(v)
        edges.append(best)
        total+=w
    return edges,round(total,2)

mst_edges,mst_total=prim_mst(graph,'W')
print('MST total:',mst_total)
print('MST edges:',mst_edges)
```

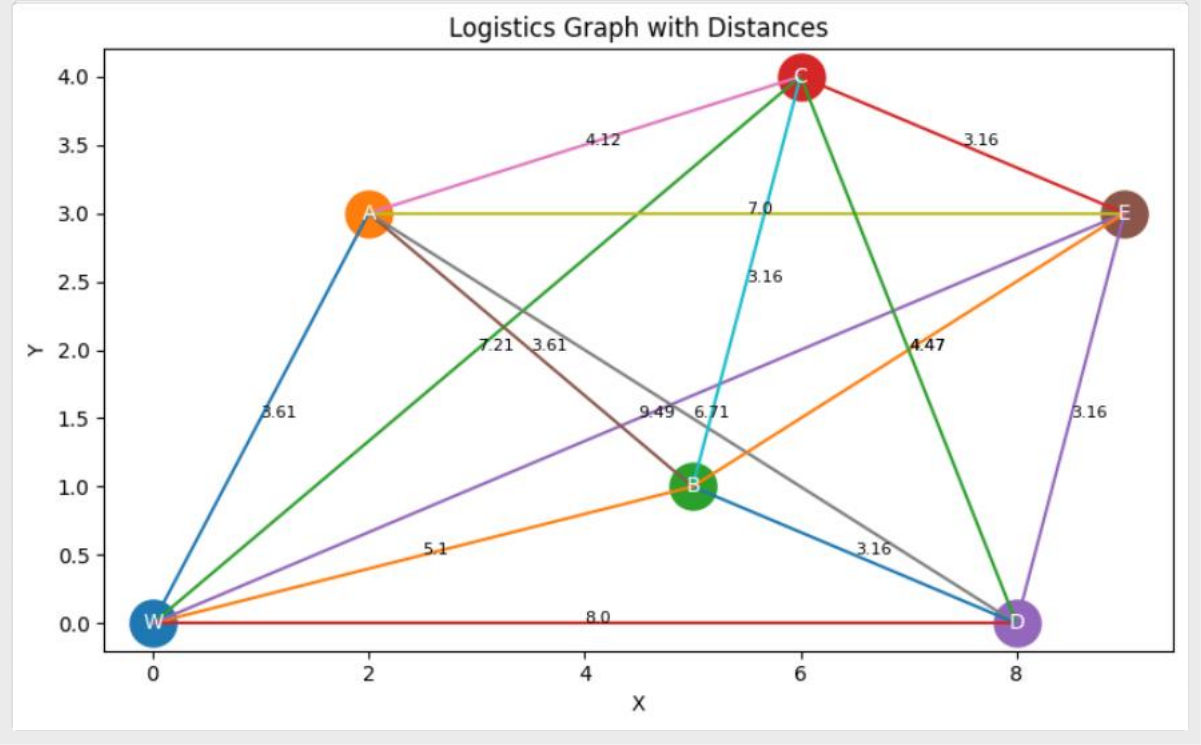
Out [15]:

```
MST total: 16.7
MST edges: [('W', 'A', 3.61), ('A', 'B', 3.61), ('B', 'C', 3.16), ('C', 'E', 3.16), ('B', 'D', 3.16)]
```

In [17]:

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 5))
for node in nodes:
    x, y =coords[node]
    ax.scatter(x, y, s=500)
    ax.text(x, y, str(node), fontsize=10, ha='center', va='center', color='white')

for i in range(len(nodes)):
    for j in range(i +1, len(nodes)):
        u =nodes[i]
        v =nodes[j]

        x1, y1 =coords[u]
        x2, y2 =coords[v]
        ax.plot([x1, x2], [y1, y2])
        mx, my = (x1 + x2) / 2, (y1 + y2) / 2
        ax.text(mx, my, str(dist_matrix[u][v]), fontsize=8)

ax.set_title("Logistics Graph with Distances")
ax.set_xlabel("X")
ax.set_ylabel("Y")
plt.tight_layout()
plt.show()
```

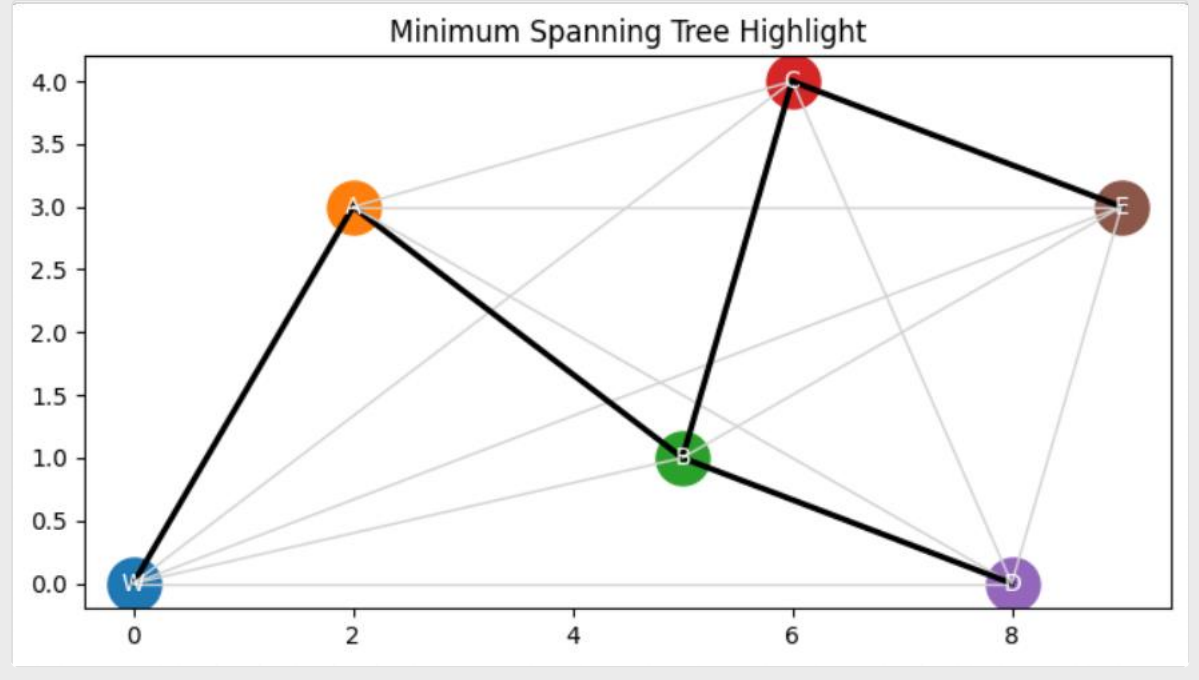
Out [17]:



In [19]:

```
fig,ax=plt.subplots(figsize=(7,4))
mst_set={(min(u,v),max(u,v)) for u,v,w in mst_edges}

for i in range(len(nodes)):
    for j in range(i+1,len(nodes)):
        u,v=nodes[i],nodes[j]
        x1,y1=coords[u]
        x2,y2=coords[v]
        if (min(u,v),max(u,v)) in mst_set:
            ax.plot([x1,x2],[y1,y2],linewidth=2.5,color='black')
        else:
            ax.plot([x1,x2],[y1,y2],linewidth=1,color='lightgray')

for n in nodes:
    x,y=coords[n]
    ax.scatter(x,y,s=500)
    ax.text(x,y,str(n),ha='center',va='center',color='white')

ax.set_title('Minimum Spanning Tree Highlight')
plt.tight_layout()
plt.show()
```

Out [19]:



6. TSP Using Held-Karp Dynamic Programming

We solve the optimal tour for the small set of customer locations.

In [20]:

```
def tsp_held_karp(locations):
    start=locations[0]
    others=locations[1:]
    m=len(others)
    dp={}
    parent={}
    for i,node in enumerate(others):
        mask=1<<i
        dp[(mask,i)]=dist_matrix[start][node]
        parent[(mask,i)]=None
    for mask in range(1,1<<m):
        for j in range(m):
            if not(mask>>j)&1:
                continue
            if mask==(1<<j):
                continue
            pm=mask^(1<<j)
            best=(math.inf,None)
            for k in range(m):
                if (pm>>k)&1:
                    cand=dp[(pm,k)]+dist_matrix[others[k]][others[j]]
                    if cand<best[0]:
                        best=(cand,k)
            dp[(mask,j)]=best[0]
            parent[(mask,j)]=best[1]
    full=(1<<m)-1
    best_end=(math.inf,None)
    for j in range(m):
        cand=dp[(full,j)]+dist_matrix[others[j]][start]
        if cand<best_end[0]:
            best_end=(cand,j)
    mask=full
    j=best_end[1]
    rev=[others[j]]
    while parent[(mask,j)] is not None:
        k=parent[(mask,j)]
        mask^=(1<<j)
        j=k
```

```
        rev.append(others[j])
    route=[start]+list(reversed(rev))+[start]
    return route,round(best_end[0],2)

tsp_route,tsp_cost=tsp_held_karp(['W','A','B','C','D','E'])
print('TSP route:',tsp_route)
print('TSP cost:',tsp_cost)
```

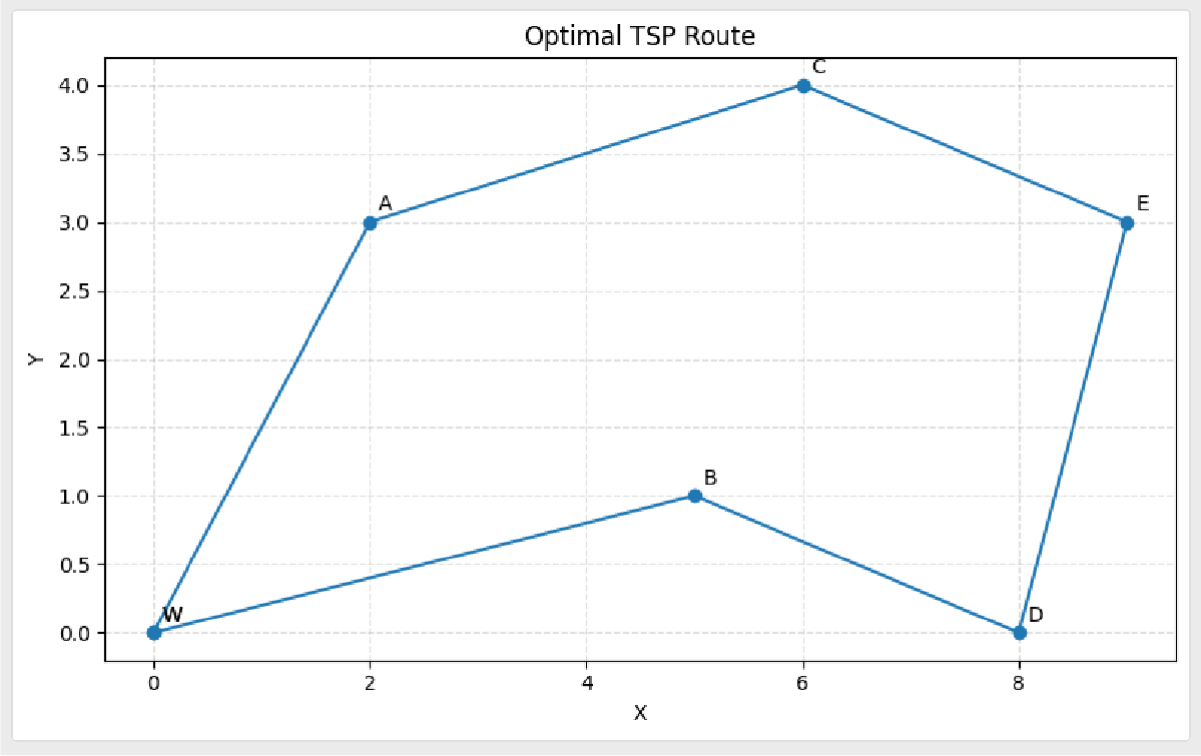
Out [20]:

```
TSP route: ['W', 'B', 'D', 'E', 'C', 'A', 'W']
TSP cost: 22.31
```

In [21]:

```
fig,ax=plt.subplots(figsize=(8,5))
xs=[coords[n][0] for n in tsp_route]
ys=[coords[n][1] for n in tsp_route]
ax.plot(xs,ys,marker='o')
for n in tsp_route:
    ax.text(coords[n][0]+0.08,coords[n][1]+0.08,n)
ax.set_title('Optimal TSP Route')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.grid(True,linestyle='--',alpha=0.4)
plt.tight_layout()
plt.show()
```

Out [21]:



7. Profiling and Comparison Plots

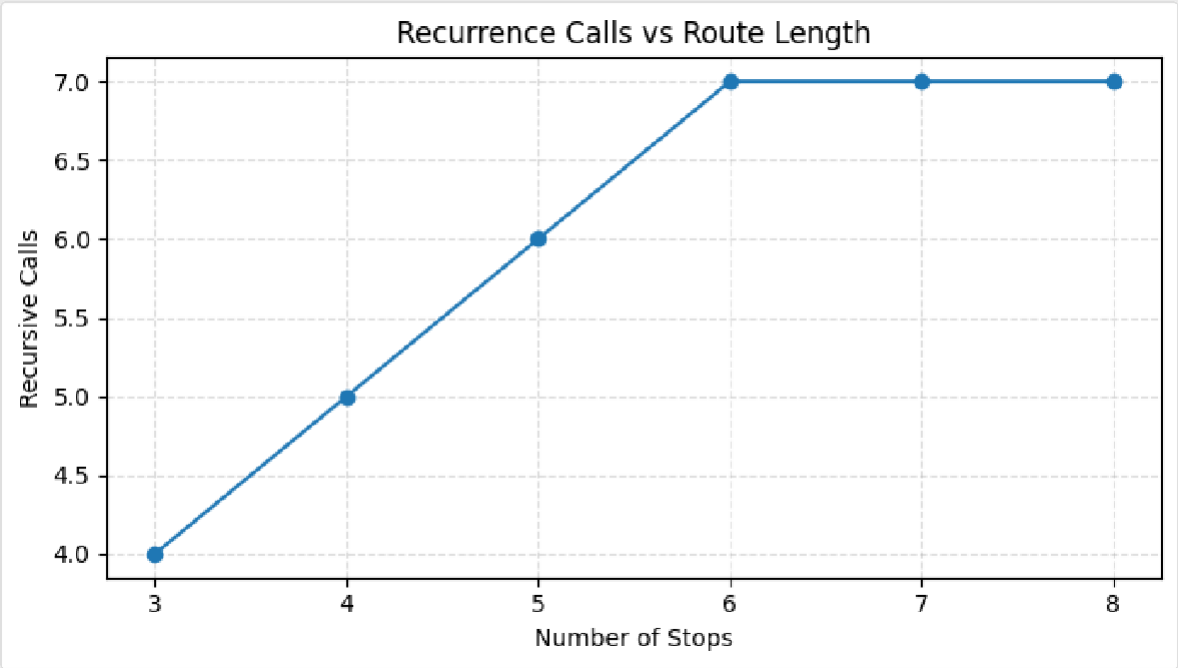
We compare runtime growth for recurrence and TSP as the problem size increases.

In [22]:

```
def route_cost_count(route):
    c={'calls':0}
    def f(r):
        c['calls']+=1
        if len(r)<2:
            return 0
        return dist_matrix[r[0]][r[1]]+f(r[1:])
    val=f(route)
```

```
    return val,c['calls']

sizes=range(3,9)
rec_calls=[]
for n in sizes:
    sub=['W']+list('ABCDE'[:n-1])
    r=sub+['W']
    _,calls=route_cost_count(r)
    rec_calls.append(calls)

tsp_sizes=range(4,8)
tsp_times=[]
for n in tsp_sizes:
    locs=['W']+list('ABCDE'[:n-1])
    st=time.perf_counter()
    tsp_held_karp(locs)
    tsp_times.append(time.perf_counter()-st)

fig,ax=plt.subplots(figsize=(7,4))
ax.plot(list(sizes),rec_calls,marker='o')
ax.set_title('Recurrence Calls vs Route Length')
ax.set_xlabel('Number of Stops')
ax.set_ylabel('Recursive Calls')
ax.grid(True,linestyle='--',alpha=0.4)
plt.tight_layout()
plt.show()
```

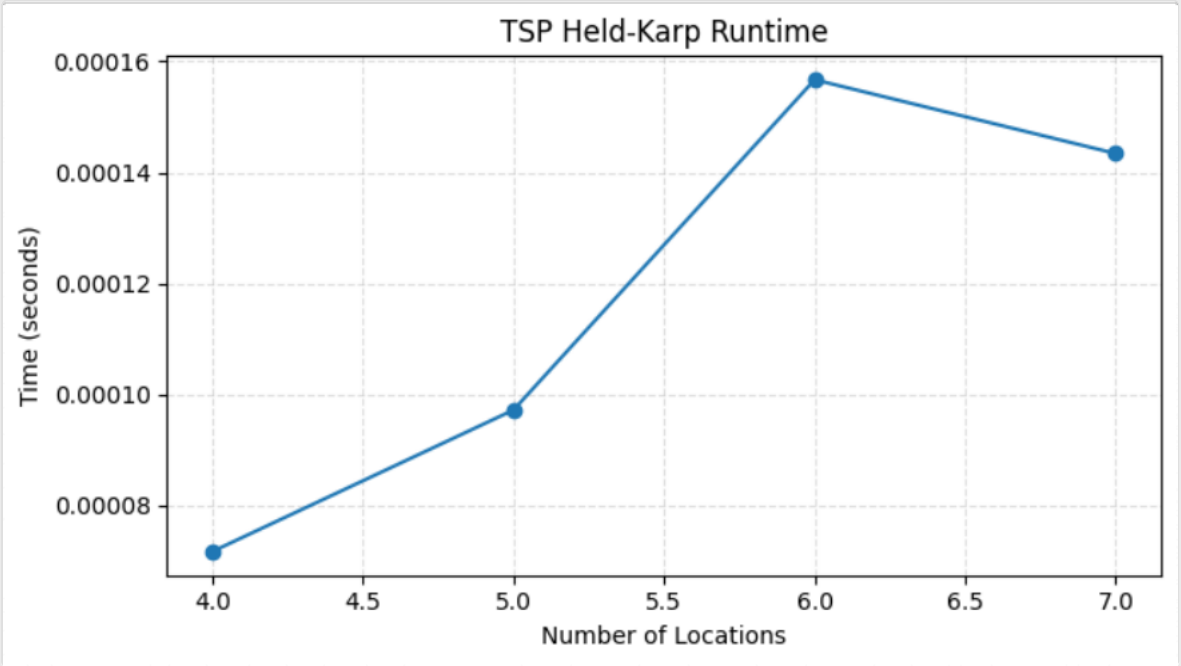
Out [22]:



In [23]:

```
fig,ax=plt.subplots(figsize=(7,4))
ax.plot(list(tsp_sizes),tsp_times,marker='o')
ax.set_title('TSP Held-Karp Runtime')
ax.set_xlabel('Number of Locations')
ax.set_ylabel('Time (seconds)')
ax.grid(True,linestyle='--',alpha=0.4)
plt.tight_layout()
plt.show()
```

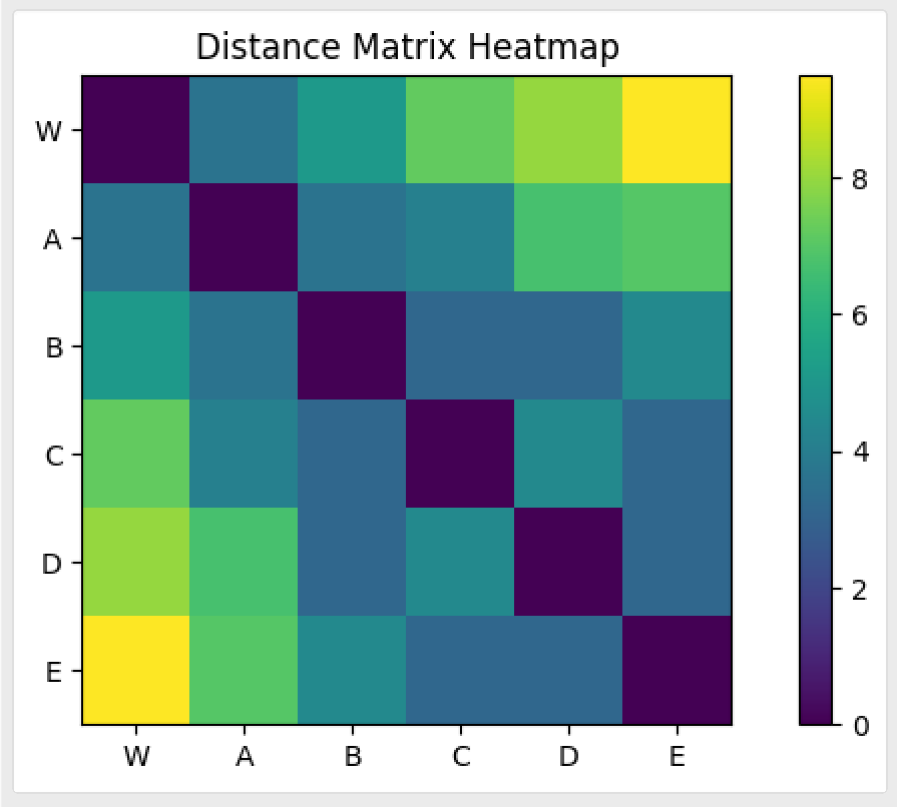
Out [23]:



In [24]:

```
fig,ax=plt.subplots(figsize=(7,4))
ax.imshow([[dist_matrix[u][v] for v in nodes] for u in nodes])
ax.set_xticks(range(len(nodes)))
ax.set_yticks(range(len(nodes)))
ax.set_xticklabels(nodes)
ax.set_yticklabels(nodes)
ax.set_title('Distance Matrix Heatmap')
plt.colorbar(ax.images[0],ax=ax)
plt.tight_layout()
plt.show()
```

Out [24]:



8. Summary Table and Observations

In [25]:

```
summary=[
    ['Greedy',v,'Fast selection by ratio'],
    ['DP',dp_v,'Better value under capacity'],
    ['Dijkstra',round(shortest['E'],2),'Shortest path from warehouse'],
    ['MST',mst_total,'Low-cost route structure'],
    ['TSP',tsp_cost,'Optimal closed tour']
]

print(f"{'Method':<12}{'Result':<12}{'Note'}")
for row in summary:
    print(f"{row[0]:<12}{str(row[1]):<12}{row[2]}")
```

Out [25]:

```
Method      Result      Note
Greedy      E           Fast selection by ratio
DP          109         Better value under capacity
Dijkstra    9.49        Shortest path from warehouse
MST         16.7        Low-cost route structure
TSP         22.31       Optimal closed tour
```

Key points

- Greedy is fast but may miss the best parcel combination.
- DP improves the delivery plan under capacity.
- Dijkstra finds shortest routes from the warehouse.
- MST gives a compact routing backbone.
- TSP produces the best overall tour for small location counts, but runtime grows quickly.

https://github.com/KLAWS5/graph-algo-mini-project--Md.-Anas-khan-